

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МОЭВМ

ОТЧЕТ
по лабораторной работе №3
по дисциплине «Построение и анализ алгоритмов»
ТЕМА: КОММИВОЯЖЁР: ДИНАМИЧЕСКОЕ ПРОГРАМИРОВАНИЕ

Студентка гр. 4343 _____ Геращенко М.Д.

Преподаватель _____ Жангиров Т.Р.

Санкт-Петербург

2026

Задание.

Даны две строки S и T , состоящие из латинских букв. Необходимо определить минимальное количество операций, требуемых для преобразования строки S в строку T . Разрешены операции вставки одного символа, удаления одного символа и замены одного символа на другой.

В более общем варианте каждая операция имеет собственную положительную стоимость: стоимость замены, вставки и удаления. Тогда требуется найти минимальную суммарную стоимость преобразования.

Во второй задаче требуется не только вычислить минимальную стоимость, но и восстановить последовательность операций: M — совпадение символов, R — замена, I — вставка, D — удаление.

Описание алгоритма

1. Считать входные строки и, при необходимости, стоимости операций.
2. Создать таблицу dp или две строки таблицы для экономии памяти.
3. Заполнить базовые случаи: преобразование пустой строки в префикс другой строки и наоборот.
4. Для каждой пары префиксов вычислить минимальную стоимость через операции M , R , I , D .
5. Для варианта 7 перед операциями R и D проверить, разрешены ли они для текущего символа первой строки.
6. Если требуется редакционное предписание, сохранить предыдущую клетку и выполненную операцию в таблице `parent`.
7. Вывести минимальное расстояние или восстановленную последовательность операций.

Выполнение работы.

В ходе выполнения лабораторной работы была разработана программа для вычисления редакционного расстояния между двумя строками. В качестве основного метода было выбрано динамическое программирование,

так как задача обладает свойством оптимальной подструктуры: оптимальное преобразование двух строк можно получить на основе оптимальных преобразований их префиксов.

На первом этапе были определены входные данные: две строки S и T . Для классической задачи считалось, что операции вставки, удаления и замены имеют одинаковую стоимость, равную единице. Для расширенной постановки дополнительно учитывались стоимости операций `replace`, `insert` и `delete`.

Далее была введена динамическая таблица dp . Элемент $dp[i][j]$ хранит минимальную стоимость преобразования первых i символов строки S в первые j символов строки T . Базовые значения таблицы соответствуют преобразованию пустой строки в непустую и наоборот: для получения j символов требуется j вставок, а для удаления i символов требуется i удалений.

При заполнении таблицы для каждой пары индексов рассматривались возможные операции. Если текущие символы строк совпадали, использовался переход M без увеличения стоимости. Если символы отличались, выбирался минимум между удалением символа, вставкой символа и заменой одного символа на другой.

Для задачи, в которой требуется вывести только расстояние Левенштейна, была выполнена оптимизация памяти. Вместо хранения всей таблицы использовались только две строки: `previous` и `current`. Это возможно, так как при вычислении очередной ячейки нужны только значения из предыдущей строки и уже вычисленное значение слева в текущей строке.

Для задачи восстановления редакционного предписания дополнительно использовалась таблица `parent`. В ней сохранялась информация о том, из какой клетки был выполнен переход и какая операция была выбрана: M , R , I или D . После заполнения таблицы путь восстанавливался обратным проходом от $dp[n][m]$ к $dp[0][0]$.

Для варианта 7 были добавлены ограничения на операции над проклятыми элементами первой строки. Перед выполнением операций замены и удаления проверялось, не относится ли текущий символ к списку запрещённых индексов. В подварианте 7а проклятый символ U разрешалось удалять, но не заменять. В подварианте 7б проклятый символ Z разрешалось заменять, но не удалять.

Правильность работы алгоритма была проверена на тестовых данных. Для строк pedestal и stien программа получила расстояние Левенштейна, равное 7. Для строк entrance и reenterable при единичных стоимостях операций была получена минимальная стоимость 5 и восстановлено редакционное предписание IMIMMIMMRRM.

Таким образом, в ходе выполнения работы были реализованы три варианта программы: вычисление обычного расстояния Левенштейна, вычисление взвешенного редакционного расстояния и восстановление редакционного предписания минимальной стоимости.

Вывод.

В ходе лабораторной работы был изучен алгоритм нахождения расстояния Левенштейна. Для решения использовалось динамическое программирование, позволяющее эффективно вычислять минимальное количество операций преобразования одной строки в другую. Была рассмотрена оптимизация памяти, благодаря которой для задачи вычисления только расстояния достаточно хранить две строки таблицы.

Также было рассмотрено взвешенное редакционное расстояние и восстановление редакционного предписания. Для варианта 7 были добавлены проверки, запрещающие замену и удаление проклятых элементов первой строки, кроме специальных случаев для символов U и Z.